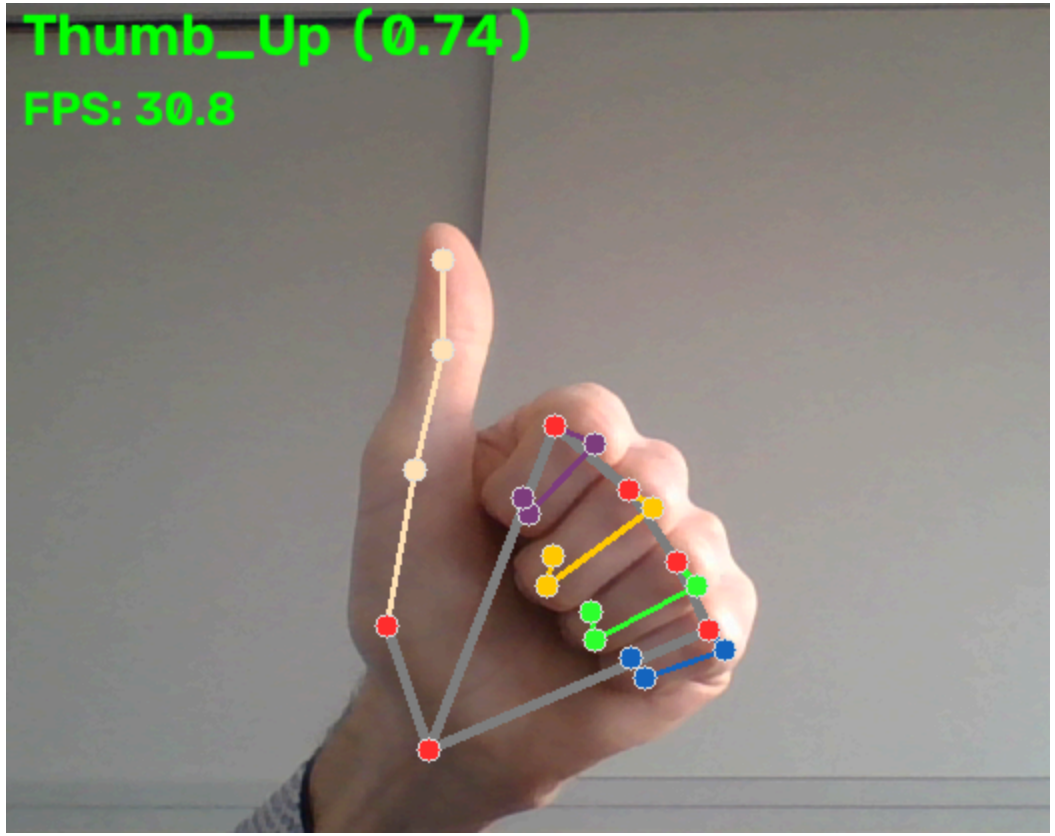
Assistive Technologies Platformen

Hand Gesture Recognition

FH University of Applied Sciences
TECHNIKUM WIEN

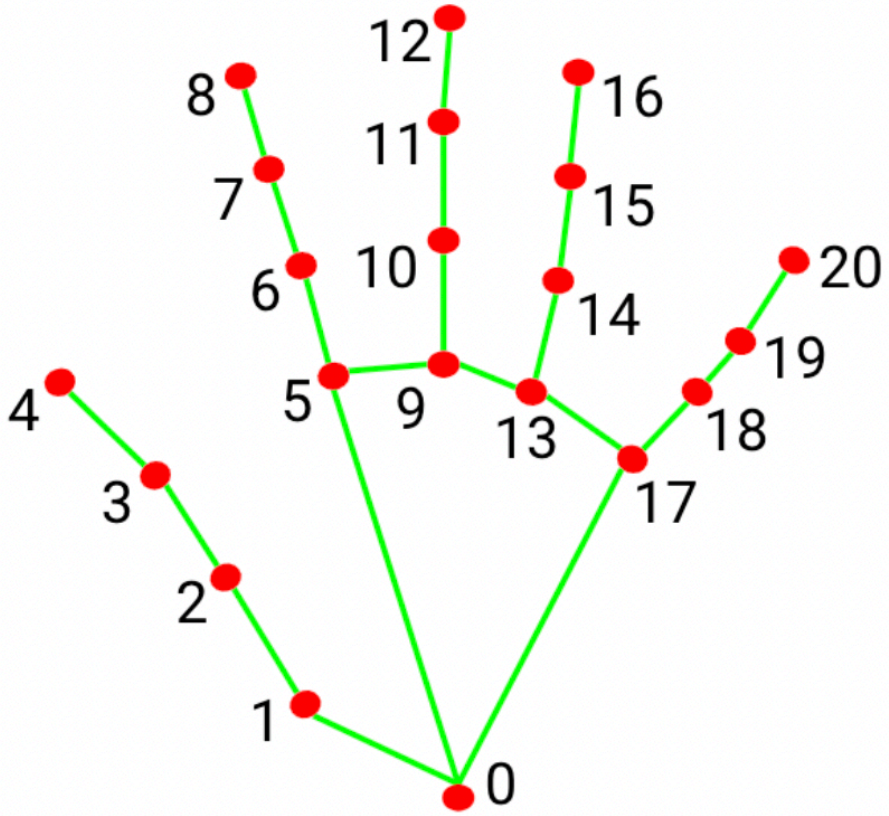
Example result

The finished webcam window shows detected hand landmarks and connections, the predicted gesture with its score, and the FPS overlay.



Hand landmarks of mediapipe GestureRecognizer

Illustration of detected hand landmarks including indices and label:



0. WRIST
1. THUMB_CMC
2. THUMB_MCP
3. THUMB_IP
4. THUMB_TIP
5. INDEX_FINGER_MCP
6. INDEX_FINGER_PIP
7. INDEX_FINGER_DIP
8. INDEX_FINGER_TIP
9. MIDDLE_FINGER_MCP
10. MIDDLE_FINGER_PIP
11. MIDDLE_FINGER_DIP
12. MIDDLE_FINGER_TIP
13. RING_FINGER_MCP
14. RING_FINGER_PIP
15. RING_FINGER_DIP
16. RING_FINGER_TIP
17. PINKY_MCP
18. PINKY_PIP
19. PINKY_DIP
20. PINKY_TIP

What we are building

For every frame from the webcam, the program will:

1. Read a frame with OpenCV.
2. Convert the frame from OpenCV's BGR channel order to RGB.
3. Ask MediaPipe's Gesture Recognizer to process the frame.
4. Draw hand landmarks, their connections, the gesture name, and its score.
5. Show the result and an estimated frame rate.

The recognizer runs in **video mode**, so each frame also needs a timestamp that increases over time. See the MediaPipe Gesture Recognizer Python guide for the task workflow and model options.

API references

Use the official API documentation when you need the full list of parameters, return values, and supported properties:

Library	Relevant API descriptions
OpenCV	Video I/O (`VideoCapture`), color conversion (`cvtColor`), drawing (`putText`), windows and keyboard input
MediaPipe	Gesture Recognizer, options, result, image, hand landmark connections

These references are useful beyond the calls shown here: follow a class or function's linked methods, properties, and parameter types to discover related capabilities.

Investigate other API functions and properties

1. In the official API reference, open the class or module for an object used in the tutorial and scan its **Methods**, **Attributes**, and parameter descriptions.
2. In Python, use `help()` for documentation and `dir()` to list names available on an object. For example:

```
help(cv2.VideoCapture.read)
dir(result)
help(type(result))
```

3. In an editor or notebook, type `.` after an object (for example, `result.`) to inspect completions, and hover over a name for its signature and documentation. Try this with `cap`, `result`, or `recognizer`.

`dir()` shows names, not necessarily a complete explanation; confirm behavior and argument details in the linked API reference. Some native-library functions may not expose a full Python signature to `help()`.

Before you start

Create a file named `requirements.txt` with the following contents:

```
opencv-python==5.0.*
mediapipe==1.1.*
```

Then install the project dependencies:

```
pip install -r requirements.txt
```

The model bundle must be available at `models/gesture_recognizer.task` and can be downloaded from the Gesture Recognizer models section.

Step 1 — Import the tools and select the model

Add these imports and settings at the top of the program:

```
import time

import cv2
import mediapipe as mp
from mediapipe.tasks.python.vision.gesture_recognizer_result import (
    GestureRecognizerResult,
)

model_path = "models/gesture_recognizer.task"
```

`cv2` handles webcam capture (Video I/O API), color conversion (image-processing API), text (drawing API), and windows (HighGUI API). `mediapipe` provides the Gesture Recognizer. `time` supplies timestamps and the clock used later to estimate FPS. The model path points to the downloaded task bundle.

Step 2 — Configure the gesture recognizer

Select the MediaPipe Tasks APIs, then create options in video mode:

```
BaseOptions = mp.tasks.BaseOptions
GestureRecognizer = mp.tasks.vision.GestureRecognizer
GestureRecognizerOptions = mp.tasks.vision.GestureRecognizerOptions
RunningMode = mp.tasks.vision.RunningMode

options = GestureRecognizerOptions(
    base_options=BaseOptions(model_asset_path=model_path),
    running_mode=RunningMode.VIDEO,
)
```

`BaseOptions` tells MediaPipe which trained model to load. `GestureRecognizerOptions` configures the task, and `RunningMode` selects how images are supplied. We use synchronous `recognize_for_video()`: it returns the result for the current frame before the loop continues.

Step 3 — Open the webcam

Create a capture object before starting the processing loop:

```
camera_index = 0  # Change this to the webcam index for your computer.
cap = cv2.VideoCapture(camera_index)

if not cap.isOpened():
    raise RuntimeError(f"Could not open webcam {camera_index}")
```

OpenCV's `VideoCapture` API opens a camera device. Index `0` is often the default camera, but the correct index varies. Checking `isOpened()` catches a bad index or unavailable camera before the loop begins.

Step 4 — Create camera loop and read one frame at a time

Create a loop, read the next frame with `VideoCapture.read()` and show it in a GUI:

```
while True:
    success, frame = cap.read()

    if not success:
        print("Could not read a webcam frame")
        break

    cv2.imshow("Gesture recognition", frame)
```

`read()` returns two values:

- `success` : whether a frame was captured.
- `frame` : the image as a NumPy array when capture succeeds.

Stop on a failed read rather than sending an empty frame to MediaPipe. A continuously failing camera would otherwise make the loop spin without producing useful output.

Step 5 — In the loop: Let the user stop the program

Check for Escape after showing each frame, then always release the camera:

```
    if cv2.waitKey(1) & 0xFF == 27:
        break

# Outside the loop:
cap.release()
cv2.destroyAllWindows()
```

OpenCV's HighGUI API documents `waitKey`, which gives the window time to update and reads a key press. The `& 0xFF` pattern makes the key comparison work across platforms; `27` is Escape. Releasing the camera and closing OpenCV windows are important cleanup steps.

Step 6: Test camera opening and capturing

Test the code opening the webcam and capturing the frames in a loop:

```
python gesture-recognition-step-by-step.py
```

You should be able to stop the program with `ESC` .

Step 7 — Know the image formats

An image is pixel data, but APIs may expect different channel orders or representations:

Name	What it means here
BGR	Three-channel NumPy image in Blue, Green, Red order. This is what OpenCV camera capture returns by default.
RGB	Three-channel NumPy image in Red, Green, Blue order. This is the channel order we pass to MediaPipe.
SRGB	The MediaPipe image-format label for standard RGB image data. It describes how to interpret the pixel bytes; it does not reorder them.
Grayscale / BGRA / RGBA	Other common layouts: one intensity channel, or four channels with alpha. They are not the three-channel format used in this example.

Color space and **channel order** are related but different ideas. Here, the practical issue is that the same three bytes are interpreted in different orders by OpenCV and MediaPipe.

Step 8 — In the loop: convert BGR to RGB

Convert the captured frame before wrapping it in a MediaPipe `Image`:

```
    rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    mp_image = mp.Image(
        image_format=mp.ImageFormat.SRGB,
        data=rgb_frame,
    )
```

OpenCV's `frame` is BGR, while the MediaPipe `Image` is declared as `SRGB` and therefore expects RGB-ordered data. The `SRGB` label does **not** perform conversion for us; OpenCV's `cvtColor` changes the channel order.

Without the conversion, red and blue are swapped when the image is interpreted. That can give the recognizer incorrectly colored input and makes the format declaration inaccurate.

Step 9 — In the loop: give each frame a timestamp

For every frame, obtain milliseconds from a monotonic clock:

```
    timestamp_ms = time.monotonic_ns() // 1_000_000
```

`monotonic_ns()` is not affected by changes to the computer's wall clock. Dividing nanoseconds by one million converts the value to milliseconds, as expected by the video API.

Video-mode timestamps must increase from frame to frame. Do not use a constant timestamp, and avoid wall-clock time such as `time.time()` for measuring intervals.

Step 10 — In the loop: recognize a gesture

Create the recognizer once, **outside the camera loop**:

```
with GestureRecognizer.create_from_options(options) as recognizer:
    while True:
    ...
```

Then recognize and display each frame with function `display_result` (must be defined before):

```
    ...
    # Inside the webcam loop, after creating mp_image and timestamp_ms:
    result = recognizer.recognize_for_video(mp_image, timestamp_ms)
    # Function display_result must be defined outside loop
    display_result(frame, result, fps)
```

The `with` block manages the recognizer's native resources. Creating it once avoids reloading the model for every frame. The `GestureRecognizer` API describes `recognize_for_video`; it processes one image and returns a `GestureRecognizerResult`. The result contains gesture classifications and hand landmarks. Each detected hand has its own entries in these lists.

Step 11 — In display_result: draw landmarks and connections

In MediaPipe 1.1.1, use the Tasks vision drawing tools and hand connection definitions:

```
def display_result(frame, result: GestureRecognizerResult, fps: float) -> None:
    annotated_frame = frame.copy()

    for hand_landmarks in result.hand_landmarks:
        mp_drawing.draw_landmarks(
            annotated_frame,
            hand_landmarks,
            mp_hands.HAND_CONNECTIONS,
            mp_drawing_styles.get_default_hand_landmarks_style(),
            mp_drawing_styles.get_default_hand_connections_style(),
        )
```

Each detected hand has 21 normalized landmarks. `HandLandmarksConnections` describes which landmark pairs form the hand skeleton. The drawing utilities API and drawing styles API provide the default point and line styles.

Step 12 — In display_result: add the gesture label

Annotate the top-ranked gesture label on the frame when a classification is available:

```
    ...
    if result.gestures and result.gestures[0]:
        top_gesture = result.gestures[0][0]
        label = f"{top_gesture.category_name} ({top_gesture.score:.2f})"
        cv2.putText(
            annotated_frame, label, (10, 30),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2,
        )
```

`category_name` is the predicted gesture label. `score` is the model's classification score; formatting it with `.2f` shows two decimal places. See the result API and its linked category type for other returned properties.

The nested list is organized by detected hand, then by category. This snippet displays the highest-ranked category for the **first** hand; a multi-hand application can loop through the hands and their categories.
OpenCV's `putText` API uses BGR color order too;

Step 13 — In the loop: calculate the frame rate

To estimate the displayed stream rate, measure the time between successful frame reads:

```
    ...
    previous_frame_time = None

    # After a successful cap.read():
    frame_time = time.perf_counter()
    fps = 0.0
    if previous_frame_time is not None:
        elapsed_seconds = frame_time - previous_frame_time
        if elapsed_seconds > 0:
            fps = 1.0 / elapsed_seconds
    previous_frame_time = frame_time
```

`perf_counter()` is a high-resolution monotonic clock for measuring elapsed time. One frame divided by the seconds between frames gives frames per second: **FPS = 1 / elapsed_seconds**.

The first frame has no previous timestamp, so its FPS starts at `0.0`. This is an interval estimate, so it can fluctuate from frame to frame; smoothing multiple intervals can make the display steadier.

Step 14 — In display_result: draw the FPS and show the window

Add an FPS overlay on every frame, even if no hand is detected:

```
    ...
    cv2.putText(
        annotated_frame,
        f"FPS: {fps:.1f}",
        (10, 65),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.8,
        (0, 255, 0),
        2,
    )
    cv2.imshow("Gesture recognition", annotated_frame)
```

This FPS estimates the rate at which successful camera frames reach the loop. It includes recognition and display work between reads, as well as camera timing; it is not a benchmark of the model alone. OpenCV's HighGUI API documents `imshow` and the related window and keyboard functions.

Complete program (1/6): setup

This first part combines the imports and recognizer configuration:

```
import time

import cv2
import mediapipe as mp
from mediapipe.tasks.python.vision.gesture_recognizer_result import (
    GestureRecognizerResult,
)

model_path = "models/gesture_recognizer.task"
BaseOptions = mp.tasks.BaseOptions
GestureRecognizer = mp.tasks.vision.GestureRecognizer
GestureRecognizerOptions = mp.tasks.vision.GestureRecognizerOptions
RunningMode = mp.tasks.vision.RunningMode
mp_hands = mp.tasks.vision.HandLandmarksConnections
mp_drawing = mp.tasks.vision.drawing_utils
mp_drawing_styles = mp.tasks.vision.drawing_styles

options = GestureRecognizerOptions(
    base_options=BaseOptions(model_asset_path=model_path),
    running_mode=RunningMode.VIDEO,
)
```

Complete program (2/6): draw the landmarks

Add this function after the setup. It draws all detected hands:

```
def display_result(frame, result: GestureRecognizerResult, fps: float) -> None:
    annotated_frame = frame.copy()

    for hand_landmarks in result.hand_landmarks:
        mp_drawing.draw_landmarks(
            annotated_frame,
            hand_landmarks,
            mp_hands.HAND_CONNECTIONS,
            mp_drawing_styles.get_default_hand_landmarks_style(),
            mp_drawing_styles.get_default_hand_connections_style(),
        )
```

Complete program (3/6): add the overlays

Continue the `display_result` function with the gesture and FPS overlays:

```
    if result.gestures and result.gestures[0]:
        gesture = result.gestures[0][0]
        label = f"{gesture.category_name} ({gesture.score:.2f})"
        cv2.putText(
            annotated_frame, label, (10, 30),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2,
        )

    cv2.putText(
        annotated_frame, f"FPS: {fps:.1f}", (10, 65),
        cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2,
    )
    cv2.imshow("Gesture recognition", annotated_frame)
```

Complete program (4/6): open the webcam

Start the camera loop below the display function. Continue the indented loop body on the next slides:

```
camera_index = 0
cap = cv2.VideoCapture(camera_index)
if not cap.isOpened():
    raise RuntimeError(f"Could not open webcam {camera_index}")

previous_frame_time = None
try:
    with GestureRecognizer.create_from_options(options) as recognizer:
        while True:
```

Complete program (5/6): capture and measure

Continue inside the `while True:` loop at the same indentation:

```
        success, frame = cap.read()
        if not success:
            print("Could not read a webcam frame")
            break

        frame_time = time.perf_counter()
        fps = 0.0
        if previous_frame_time is not None:
            elapsed_seconds = frame_time - previous_frame_time
            if elapsed_seconds > 0:
                fps = 1.0 / elapsed_seconds
        previous_frame_time = frame_time
```

Complete program (6/6): recognize and clean up

Continue the same loop. The `finally` block releases resources even if capture or recognition raises an error:

```
            rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            mp_image = mp.Image(
                image_format=mp.ImageFormat.SRGB, data=rgb_frame
            )
            timestamp_ms = time.monotonic_ns() // 1_000_000
            result = recognizer.recognize_for_video(mp_image, timestamp_ms)
            display_result(frame, result, fps)

            if cv2.waitKey(1) & 0xFF == 27:
                break
finally:
    cap.release()
    cv2.destroyAllWindows()
```

Run, observe, and experiment

Run the finished script and try these small experiments:

- Change the camera index if the webcam window is empty.
- Cover or move your hand and watch the landmarks follow it.
- Try the model's supported gestures and compare labels and scores.
- Temporarily remove `cv2.cvtColor` to see why correct channel ordering matters.
- Compare the FPS while the camera is idle and while recognition is running.

Keep the task model at the configured path, and press **Escape** to close the window cleanly.